



Facultad de Ingeniería y Ciencias.

Proyecto: Detección de Fraude con Aprendizaje por Refuerzo.

Curso: Sistemas Inteligentes.

Profesora: Ricardo Gacitua.

Autor: Marco Contreras.

Fecha de entrega: 10 de julio del 2026.

1. Introducción

El **Aprendizaje por Refuerzo** es una rama de la inteligencia artificial donde un agente aprende a tomar decisiones mediante la interacción con un entorno, recibiendo recompensas o penalizaciones según sus acciones. A diferencia del aprendizaje supervisado, no se dispone de un conjunto de datos etiquetado; el agente descubre por sí mismo qué comportamientos conducen a mejores resultados a largo plazo.

En este trabajo se aborda el problema de la **detección de transacciones fraudulentas** en una plataforma de comercio electrónico (simulando el contexto de Mercado Libre). Este escenario es especialmente relevante debido al creciente volumen de operaciones digitales y la sofisticación de los estafadores. Un sistema de detección basado en reglas fijas resulta insuficiente, ya que los patrones de fraude evolucionan constantemente. Por el contrario, un agente de RL puede adaptarse dinámicamente, mejorando su desempeño con cada interacción.

La aplicación web desarrollada permite simular este entorno, visualizar el proceso de aprendizaje y comprender los conceptos fundamentales de RL de forma interactiva.

2. Modelado del Problema

Para aplicar RL a la detección de fraudes, es necesario definir los elementos clave:

2.1 Estado

El estado representa la información disponible sobre una transacción en el momento de la decisión. Se compone de cinco atributos discretos:

Atributo	Valores posibles
Monto	Bajo, Medio, Alto
Ubicación	Local, Internacional Habitual, País de Alto Riesgo
Dispositivo	Frecuente (conocido), Nuevo

Hora	Regular (horario hábil), Madrugada
Frecuencia	Normal (1 compra hoy), Alta (5 compras seguidas)

Cada combinación única de estos valores constituye un **estado**.

2.2 Acciones

El agente dispone de dos acciones posibles:

- **Aprobar:** permitir la transacción.
- **Bloquear:** rechazar la transacción.

2.3 Función de Recompensa

La recompensa es el feedback que recibe el agente después de ejecutar una acción. Su diseño es crítico, pues guía el aprendizaje hacia el comportamiento deseado.

Se definieron los siguientes valores:

Situación Real	Acción del Agente	Recompensa	Justificación
Transacción legítima	Aprobar	+3	El agente acierta y permite una compra válida.
Transacción legítima	Bloquear	-6	Falso positivo: se bloquea a un cliente real, generando insatisfacción y pérdida de venta.

Transacción fraudulenta	Bloquear	+5	El agente detecta y detiene el fraude, evitando pérdidas económicas.
Transacción fraudulenta	Aprobar	-8	Falso negativo: el fraude pasa, causando un perjuicio significativo.

Justificación de las penalizaciones:

- Se penaliza más severamente **aprobar un fraude (-8)** que **bloquear una compra legítima (-6)**, porque el impacto económico y reputacional de un fraude exitoso es mayor.
- La recompensa positiva por **bloquear un fraude (+5)** es superior a la de **aprobar una legítima (+3)**, incentivando al agente a ser más conservador en situaciones de alta incertidumbre.
- Los valores fueron ajustados empíricamente para lograr un equilibrio entre sensibilidad y especificidad.

Este esquema convierte el problema en un **Bandido Contextual**, ya que las transacciones son eventos independientes; el agente no necesita considerar secuencias de estados futuros. Por ello, se fijó el factor de descuento $\gamma = 0$.

3. Implementación Tecnológica

3.1 Algoritmo: Q-Learning

El agente utiliza el algoritmo **Q-Learning** (Sutton & Barto, 2018), que mantiene una tabla $Q(s, a)$ con el valor esperado de tomar la acción a en el estado s . La actualización sigue la ecuación de Bellman:

The diagram shows the Q-learning equation: $Q(s, a) = r(s, a) + \gamma \max_a Q(s', a)$. Blue arrows point from labels to specific parts of the equation: 'reward' points to $r(s, a)$, 'discount factor' points to γ , 'state' points to s in $Q(s, a)$, and 'action' points to a in $Q(s, a)$.

$$Q(s, a) = r(s, a) + \gamma \max_a Q(s', a)$$

- $\alpha = 0.2$ (tasa de aprendizaje): controla la velocidad de actualización del modelo.
- $\gamma = 0.0$ (factor de descuento): al ser un problema contextual, no se consideran recompensas futuras.
- $\epsilon = 0.15$ (tasa de exploración): el agente elige una acción aleatoria con probabilidad 0.15 para explorar nuevas posibilidades.

La tabla Q se persiste en un archivo JSON (q_table.json) para que el aprendizaje se mantenga entre sesiones.

3.2 Stack tecnológico

Componente	Tecnología
Backend	Python con Flask (servidor REST)
Frontend	HTML5, CSS3 (Tailwind CSS), JavaScript
Visualización	Chart.js (gráfica de recompensa)
Fórmulas matemáticas	MathJax

La comunicación entre frontend y backend se realiza mediante peticiones AJAX a los endpoints /decide, /learn, /q-table y /reset.

4. Resultados y Limitaciones

4.1 Resultados de la simulación

La aplicación permite ejecutar casos de forma manual o automática (entrenamiento de 20 pasos). A continuación se muestran capturas representativas del proceso:

Figura 1: Interfaz del simulador

Simulador de Transacciones

Monto (\$)

Bajo (Menor a \$10.000)

Ubicación

Local (Ciudad natal)

Dispositivo

Celular de siempre

Hora

De día (Horario Hábil)

Frecuencia

1 compra hoy

Naturaleza Real

Legítima (Cliente Real)

Enviar caso a evaluar por el Agente IA

Entrenar 20 pasos

Figura 2: Evolución de la recompensa acumulada

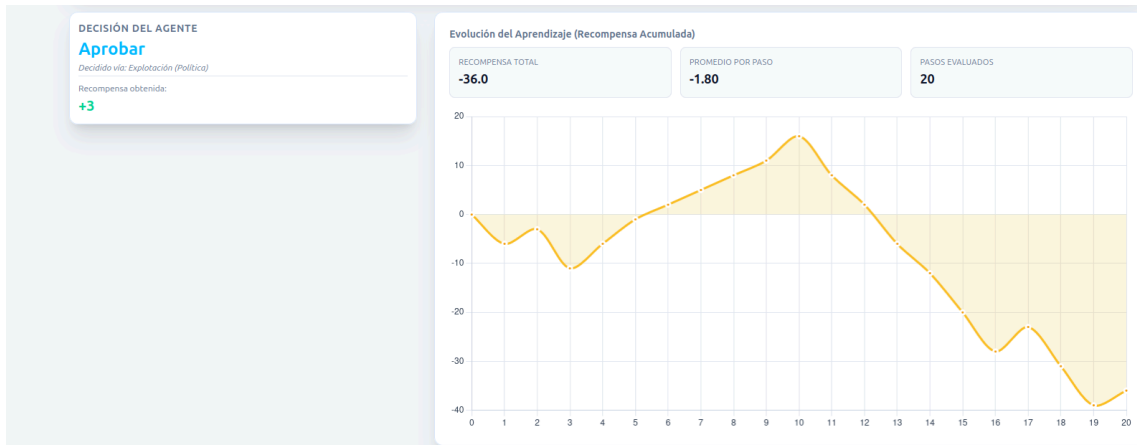
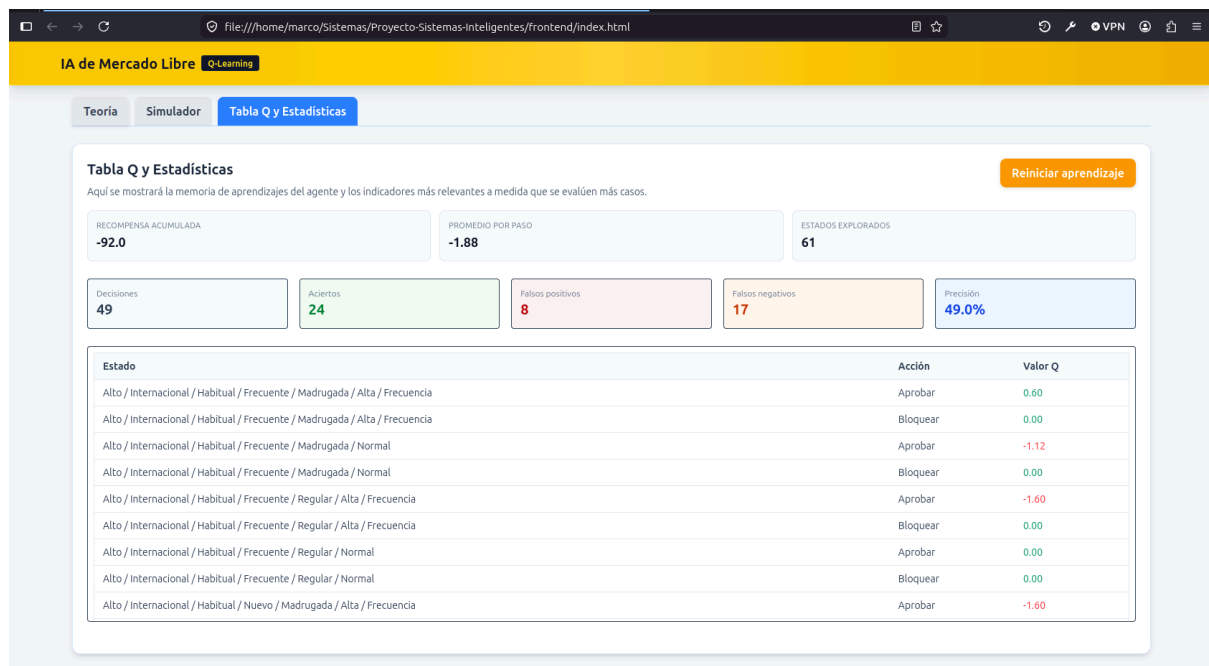


Figura 3: Tabla Q y estadísticas



Precisión del 49%: Aunque parece baja, teniendo en cuenta que el agente está en exploración ($\epsilon=0.15$) y solo ha visto 49 casos, el resultado es razonable. Con más iteraciones, la precisión debería mejorar.

Recompensa acumulada negativa (-92) indica que, en promedio, cada acción ha generado una pérdida neta. Esto se debe a que los falsos negativos (17) y falsos positivos (8) superan a los aciertos (24) en términos de penalización.

4.2 Limitaciones del enfoque

1. Q-Learning requiere que el espacio de estados sea finito y discreto. En la práctica, atributos como el monto exacto o la hora precisa son continuos. La discretización utilizada (bajo/medio/alto, etc.) simplifica el problema, pero puede perder información relevante.
2. Falta de escalabilidad para entornos con muchas combinaciones de atributos (por ejemplo, ubicaciones geográficas detalladas), la tabla Q crece exponencialmente. En escenarios masivos, se necesitarían técnicas de aproximación como **Deep Q-Networks (DQN)**, que utilizan redes neuronales para generalizar sobre estados no vistos (Mnih et al., 2015).
3. Los estafadores cambian sus tácticas con el tiempo. El agente debe reentrenarse periódicamente para adaptarse a nuevos patrones, lo que requiere un flujo continuo de datos y retroalimentación.
4. El modelo asume que las transacciones son independientes, lo cual es válido para muchos casos, pero no captura posibles correlaciones temporales (por ejemplo, un usuario que hace varias compras en un lapso corto). Un enfoque de RL completo (con $\gamma > 0$) podría modelar mejor estas dependencias.

5. Referencias

- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Li, L., Chu, W., Langford, J., & Schapire, R. E. (2010). A contextual-bandit approach to personalized news article recommendation. *Proceedings of the 19th International Conference on World Wide Web*, 661–670.
- Contreras, M. (2026). *Proyecto-Sistemas-Inteligentes* [Software de computadora]. GitHub. <https://github.com/Marco-Contreras-Progra/Proyecto-Sistemas-Inteligentes>